

PS 26052 — ANC Prototype Development

Software Implementation Task — Modules 3 to 5

Adaptive Filtering → Digital ANC Plant → Secondary-Path Identification

Build three continuous, modular implementation modules in which the validated output of each module becomes a reusable input to the next.

MODULE 1 Signal Foundation: Architecture → Generation → FIR/LTI Processing [COMPLETE]
MODULE 2 Statistical Estimation: Correlation → Wiener/MMSE Benchmark [COMPLETE]

MODULE 3 Adaptive Filtering Laboratory

↓

MODULE 4 Digital Feedforward ANC Plant

↓

MODULE 5 Secondary-Path Identification

↓

Later phases: Physical ANC → Evaluation → AI → Hybrid → Streaming → Embedded

This task continues the same implementation philosophy used for Modules 1 and 2: **configuration-driven experiments, reusable interfaces, saved artifacts, visualization, testing, and explicit acceptance conditions.**

ε	Small positive normalization constant for NLMS-type updates.
$P(z)$	Primary path from disturbance/reference source toward the error location.
$W(z)$	Adaptive ANC controller.
$S(z)$	True secondary path from controller output through the actuator/acoustic path to the error sensor.
$\hat{S}(z)$	Estimated/modelled secondary path.
$x_f[n]$	Reference signal filtered through $\hat{S}(z)$, used by filtered-x adaptation.
w_{opt}	Batch Wiener/MMSE benchmark from Module 2.

3. MODULE 3 — Adaptive Filtering Laboratory

Objective: Convert the batch Wiener/MMSE benchmark into an online adaptive filtering framework that updates coefficients sample by sample and can be evaluated against the known Wiener optimum.

Problem statement

Module 2 solved for one coefficient vector using statistics estimated from the complete dataset. That is not sufficient when the signal relationship must be learned progressively or changes over time. The task is therefore to implement a causal adaptive FIR filter that observes the current sample, forms an estimate, computes an error, and updates its coefficients repeatedly.

At each sample n :

```
reference vector  $x_n$ 
    ↓
adaptive FIR coefficients  $w[n]$ 
    ↓
 $y[n] = w^T[n] x_n$ 
    ↓
 $e[n] = d[n] - y[n]$ 
    ↓
update  $w[n] \rightarrow w[n+1]$ 
```

3A. Adaptive FIR filter structure

- Implement a reusable tapped-delay-line / causal adaptive FIR structure.
- Use the same signal conventions established by Modules 1 and 2.
- Expose coefficient history, output history, error history, and final coefficient vector.
- Support configurable filter length, initialization, sampling rate metadata, and experiment seed where applicable.
- Reject invalid dimensions, empty signals, non-finite values, and incompatible input/desired lengths.

Output:

A reusable adaptive-filter interface capable of producing $y[n]$, $e[n]$, $w[n]$, and coefficient trajectories.

3B. LMS algorithm

Implement Least Mean Squares using the instantaneous-error update:

```
 $y[n] = w^T[n] x_n$ 
 $e[n] = d[n] - y[n]$ 

 $w[n+1] = w[n] + \mu e[n] x_n$ 
```

- Support configurable μ and filter initialization.
- Store coefficient evolution and instantaneous squared error.
- Demonstrate convergence on controlled system-identification cases.
- Sweep multiple step sizes to demonstrate convergence speed, steady-state behavior, and instability boundaries.

3C. LMS convergence and learning-curve experiments

- Run the same adaptive experiment across white-noise system identification, correlated input, tonal input, and changing signal statistics.
- Produce instantaneous error and MSE/learning curves.
- Compare final adaptive coefficients and output error against the Module 2 Wiener benchmark under stationary controlled conditions.

- Record convergence behavior rather than relying only on a final scalar metric.

3D. NLMS algorithm

Implement Normalized LMS to reduce sensitivity to input power:

$$w[n+1] = w[n] + \left[\mu / (\epsilon + ||x_n||^2) \right] e[n] x_n$$

- Support configurable μ and ϵ .
- Demonstrate behavior under changing input amplitudes and signal power.
- Compare LMS and NLMS using identical signals and filter lengths.
- Report convergence, residual error, and final coefficient agreement with the Wiener benchmark.

Optional advanced benchmark

RLS may be added only after LMS and NLMS are clean and validated. It is an advanced benchmark, not a requirement for completion of this module.

Required outputs

```
adaptive_output.npy
error.npy
final_coefficients.npy
coefficient_history.npy
instantaneous_squared_error.npy
mse_learning_curve.npy
summary.json
```

Required visualizations

- Coefficient trajectories $w_k[n]$.
- Instantaneous error / squared-error evolution.
- Learning curve / MSE evolution.
- LMS vs NLMS comparison.
- Adaptive final coefficients vs Module 2 Wiener coefficients where the experiment is stationary and comparable.

Module 3 deliverable

A reproducible adaptive-filter laboratory demonstrating the complete online chain $x_n \rightarrow y[n] \rightarrow e[n] \rightarrow$ coefficient update, with LMS and NLMS implementations, convergence diagnostics, and direct comparison against the Wiener benchmark.

Acceptance conditions

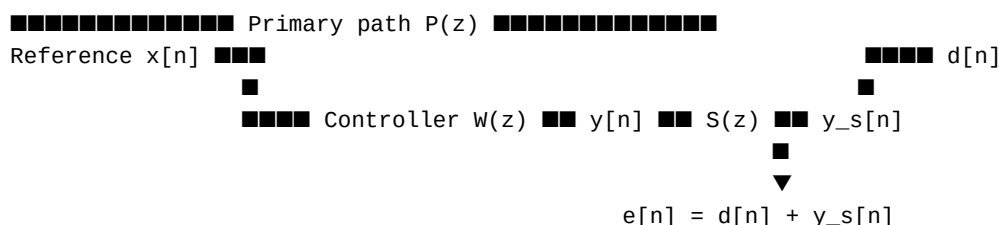
- All reusable adaptive-filter functions pass unit tests.
- LMS and NLMS converge in controlled stationary identification experiments for stable parameter choices.
- Coefficient and/or output behavior approaches the Module 2 Wiener benchmark in comparable stationary conditions.
- Changing μ produces observable and explainable changes in convergence behavior.
- NLMS remains less sensitive than ordinary LMS to controlled changes in input scale.
- One end-to-end experiment runs from configuration and saves the complete artifact set.

4. MODULE 4 — Digital Feedforward ANC Plant

Objective: Build a fully reproducible simulated ANC environment containing the primary path $P(z)$, adaptive controller $W(z)$, secondary path $S(z)$, and error signal $e[n]$.

Problem statement

Generic LMS/NLMS identifies a relationship directly between a reference and a desired signal. Real feedforward ANC is different because the controller output does not reach the error sensor directly. It propagates through the actuator/acoustic secondary path. The task is to model this complete structure in software before moving to physical hardware.



4A. Primary path

- Implement a configurable primary path $P(z)$, initially as a known FIR/LTI system.
- Generate the disturbance at the error location using a consistent plant convention.
- Support controlled path length, impulse response, delay, and gain.
- Save the primary-path representation as a reusable artifact.

4B. Adaptive controller

- Reuse the Module 3 adaptive-controller infrastructure rather than duplicating LMS/NLMS logic.
- Represent $W(z)$ as a configurable adaptive FIR controller.
- Generate controller output $y[n]$ from the reference signal.
- Track controller coefficients and error over time.

4C. Secondary path

- Implement a configurable known secondary path $S(z)$ for the digital plant.
- Propagate controller output through $S(z)$ before combining it with the primary disturbance.
- Support path delay and phase-shaping behavior through the impulse response.
- Keep the true simulated $S(z)$ available for plant evaluation, but do not silently leak it into later identification experiments where only $\hat{S}(z)$ should be used.

4D. ANC error and sign convention

The simulated error must be formed using one explicit and documented sign convention, for example:

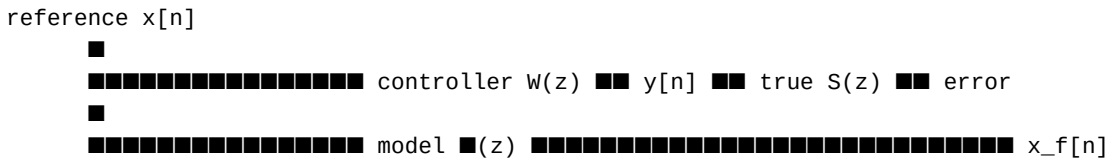
$$\begin{aligned}d[n] &= P(z) * x[n] \\y[n] &= W(z) * x[n] \\y_s[n] &= S(z) * y[n] \\e[n] &= d[n] + y_s[n]\end{aligned}$$

The controller must adapt so that the secondary-path contribution becomes approximately the negative of the primary disturbance at the error location.

4E. Algorithm progression inside the ANC plant

- No-control baseline: measure the disturbance with the controller disabled.
- LMS/NLMS baseline: demonstrate why direct adaptive filtering is not sufficient when the secondary path alters the controller-to-error relationship.
- FxLMS: implement filtered-x adaptation using a secondary-path model.
- FxNLMS: normalize the filtered-x update where appropriate for input-power robustness.

Core filtered-x structure



FxLMS updates $W(z)$ using the filtered reference $x_f[n]$.

Required digital ANC experiments

- Stationary broadband disturbance.
- Correlated / colored disturbance.
- Tonal or narrowband disturbance.
- Controlled primary/secondary-path delay cases.
- Controller-disabled baseline.
- Known-perfect secondary-path model case.

Required outputs

```

reference_x.npy
primary_disturbance_d.npy
controller_output_y.npy
secondary_path_output.npy
error_e.npy
controller_coefficients.npy
primary_path.npy
secondary_path_true.npy
secondary_path_model_used.npy
anc_summary.json
  
```

Required visualizations

- Primary disturbance vs residual error.
- No-control vs controlled error.
- Controller coefficient trajectories.
- Error power / MSE over adaptation time.
- Frequency-domain attenuation where applicable.
- Algorithm comparison: no control, LMS/NLMS baseline, FxLMS, and FxNLMS where implemented.

Module 4 deliverable

A complete reproducible digital feedforward ANC experiment in which a reference disturbance travels through a primary path, an adaptive controller generates anti-noise, the anti-noise propagates through a secondary path, and the resulting error is measured and reduced.

Acceptance conditions

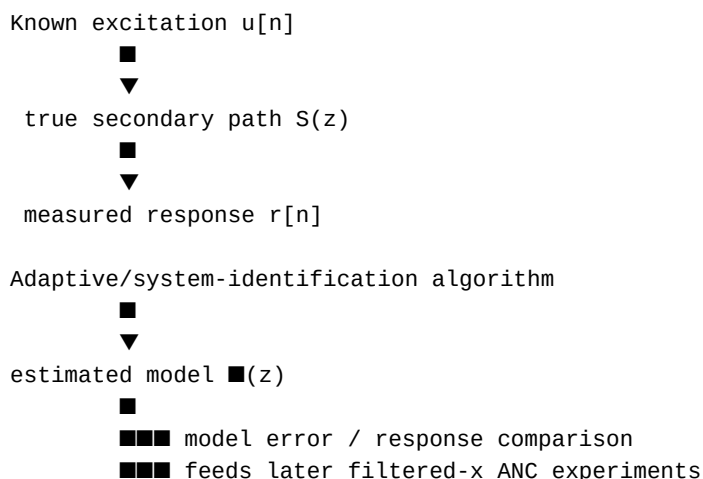
- The digital plant reproduces the documented $P(z)$, $W(z)$, $S(z)$, and error-signal relationships.
- The no-control baseline is saved and used as the reference condition.
- A correctly configured filtered-x controller reduces residual error relative to no control in controlled conditions.
- The effect of the secondary path is demonstrably included; controller output alone must not be treated as the signal arriving at the error location.
- All plant interfaces are reusable by Module 5.
- One end-to-end ANC experiment can be reproduced from configuration.

5. MODULE 5 — Secondary-Path Identification

Objective: Estimate a reusable secondary-path model $\hat{S}(z)$ from measured input/output data and quantify how modelling mismatch affects downstream ANC adaptation.

Problem statement

FxLMS/FxNLMS requires a model of the path between the controller output and the error sensor. In simulation, the true secondary path can be known. In a physical system it must be identified from measurements. Module 5 treats this as a first-class system-identification task rather than a helper script.



5A. Controlled synthetic secondary-path identification

- Create a hidden/true synthetic secondary path $S(z)$.
- Excite it with a known probe or broadband identification signal.
- Measure the resulting response.
- Use LMS, NLMS, or another validated identification method to estimate $\hat{S}(z)$.
- Compare estimated and true impulse responses without allowing the true path to enter the estimator as prior knowledge.

5B. Identification validation

- Align true and estimated impulse responses when necessary for length/delay comparison.
- Measure coefficient/impulse-response error.
- Measure MSE/RMSE or normalized response error.
- Compare magnitude and phase/frequency response where applicable.
- Store both numerical metrics and diagnostic plots.

5C. Secondary-path mismatch experiments

Deliberately test the effect of imperfect modelling by using a perturbed model:

$$\hat{S}(z) = S(z) + \Delta S(z)$$

- Gain mismatch.
- Delay mismatch.
- Impulse-response coefficient mismatch.
- Frequency-response / phase mismatch.
- Optional time-varying secondary-path cases.

Evaluate how mismatch changes convergence speed, residual error, controller bias, and stability in the digital ANC plant. This module must make the engineering consequence explicit: an inaccurate $\hat{S}(z)$ produces an inaccurate filtered reference and therefore an imperfect adaptation direction.

5D. Reusable secondary-path model artifact

- Define a stable serialization format for the identified model.
- Save impulse response, metadata, sampling rate, identification configuration, and validation metrics.
- Ensure Module 4/later ANC experiments can load $\hat{S}(z)$ without depending on experiment-specific code.

5E. Physical identification preparation

The software interface must be designed so that the synthetic path can later be replaced by measured loudspeaker → acoustic environment → error-microphone data. The later physical procedure is:

```
DAC / audio output
    ↓
speaker
    ↓
real acoustic secondary path
    ↓
error microphone
    ↓
recorded response
    ↓
identify and store  $\hat{S}(z)$ 
```

Required outputs

```
identification_excitation.npy
measured_secondary_response.npy
secondary_path_estimate.npy
secondary_path_true.npy          (synthetic validation only)
secondary_path_error.npy
secondary_path_summary.json
secondary_path_frequency_response.png
secondary_path_impulse_comparison.png
```

Required visualizations

- True vs estimated impulse response in controlled simulation.
- Impulse-response/model error.
- Magnitude response comparison.
- Phase response comparison where applicable.
- Identification convergence / error curve.
- ANC performance degradation as secondary-path mismatch increases.

Module 5 deliverable

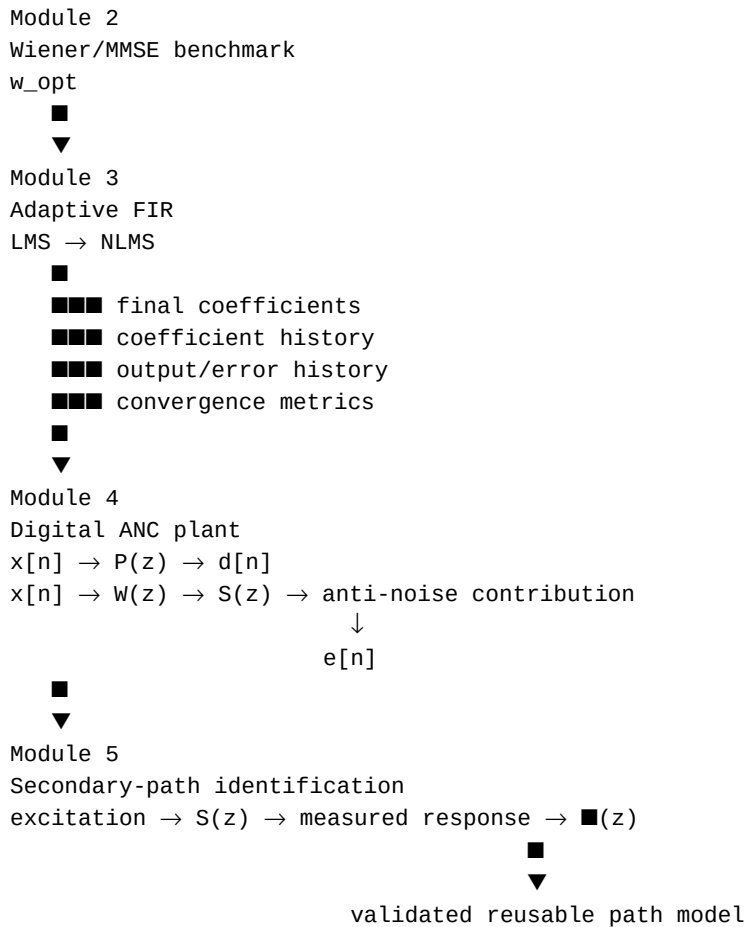
A reproducible secondary-path identification pipeline that estimates $\hat{S}(z)$, validates it against a known path in simulation, quantifies modelling mismatch, and exports a reusable secondary-path model for subsequent filtered-x ANC experiments and eventual physical calibration.

Acceptance conditions

- The estimator converges toward the hidden synthetic secondary path under controlled excitation.
- The estimated path is validated numerically and visually.

- The identified model is saved with metadata and can be reloaded by a downstream ANC experiment.
- Controlled mismatch experiments demonstrate measurable effects on ANC convergence or residual performance.
- The estimator does not use the true synthetic $S(z)$ except for post-estimation validation.
- The software interface is compatible with future measured physical secondary-path data.

6. Module Connection



7. End-of-Phase Milestone

At the end of Modules 3 to 5, the project must have crossed the boundary from **batch statistical estimation** to a working **adaptive ANC simulation framework**.

Module 2:
Known dataset → statistics → solve once → Wiener optimum

Module 3:
New samples → error → update coefficients online

Module 4:
Reference → controller → secondary path → acoustic-style error

Module 5:
Measure/identify secondary path → build ■(z) → quantify mismatch

Required phase-level evidence

- A reproducible LMS/NLMS experiment with convergence evidence and Wiener comparison.
- A reproducible digital ANC plant with explicit $P(z)$, $W(z)$, $S(z)$, and $e[n]$.
- A filtered-x ANC experiment that reduces error under controlled conditions.
- A reproducible secondary-path identification experiment producing ■(z).
- Mismatch experiments showing why secondary-path accuracy matters.
- Unit tests plus at least one end-to-end integration/acceptance test for each major module.

- Saved numerical artifacts, metadata/JSON summaries, and visual diagnostics.

8. Forward Interface

The output of Module 5 is not the end of ANC development. It becomes the interface required for later physical ANC calibration and more advanced FxLMS/FxNLMS experiments. Subsequent modules may add physical acoustic paths, standardized evaluation conditions, AI suppression, hybrid AI + adaptive DSP, robust streaming, and embedded deployment without replacing the validated interfaces established here.

Engineering rule: do not optimize or add AI before correctness is demonstrated. Every algorithm must remain independently testable, reproducible, and comparable against an explicit baseline.